

Nondeterminism (continue)

1. Given three sorted (into non-decreasing order) *size* – n arrays b_0, b_1 and b_2 , are there valid indices k_0, k_1 and k_2 such that $b_0[k_0] = b_1[k_1] = b_2[k_2]$? Create a program in our language that can find a set of such k_0, k_1 and k_2 if they exist.

- Here we use the most naïve idea: use k_0, k_1 and k_2 as pointers and scan three arrays until we find a solution or one of them reaches n .
- If we use a deterministic program, I can expect that there will be many different cases in each iteration: we are looking at three values: $b_0[k_0]$, $b_1[k_1]$ and $b_2[k_2]$, each two of them can be $=$, $<$ or $>$.
- It will be much easier if we only have two arrays, for example let's look at only $b_0[k_0]$ and $b_1[k_1]$:
 - If $b_0[k_0] = b_1[k_1]$, we are done.
 - If $b_0[k_0] < b_1[k_1]$, since arrays are sorted, increasing k_1 can only get a larger number in b_1 , should increase k_0 .
 - If $b_0[k_0] > b_1[k_1]$, like the previous case, we should increase k_1 .
- Thus, if we ignore b_2 and ignore the array size, we can come up with a partial solution to this question using a nondeterministic **do** – **od** statement immediately:

$$\mathbf{do} \ b_0[k_0] < b_1[k_1] \rightarrow k_0 := k_0 + 1 \ \square \ b_0[k_0] > b_1[k_1] \rightarrow k_1 := k_1 + 1 \ \mathbf{od}$$

- Similarly, for the other combinations of values, we can come up with some other partial solutions:

$$\mathbf{do} \ b_0[k_0] < b_2[k_2] \rightarrow k_0 := k_0 + 1 \ \square \ b_0[k_0] > b_2[k_2] \rightarrow k_2 := k_2 + 1 \ \mathbf{od}$$

$$\mathbf{do} \ b_1[k_1] < b_2[k_2] \rightarrow k_1 := k_1 + 1 \ \square \ b_1[k_1] > b_2[k_2] \rightarrow k_2 := k_2 + 1 \ \mathbf{od}$$

- Then, we can see the beauty of using a nondeterministic program: we don't need to worry about the overlapping cases, and it is very easy to combine partial solutions. We can come up with the following program:

$$K \equiv$$

$$k_0 := 0; k_1 := 0; k_2 := 0;$$

$$\mathbf{do} \ b_0[k_0] < b_1[k_1] \rightarrow k_0 := k_0 + 1$$

$$\square \ b_0[k_0] > b_1[k_1] \rightarrow k_1 := k_1 + 1$$

$$\square \ b_0[k_0] < b_2[k_2] \rightarrow k_0 := k_0 + 1$$

$$\square \ b_0[k_0] > b_2[k_2] \rightarrow k_2 := k_2 + 1$$

$$\square \ b_1[k_1] < b_2[k_2] \rightarrow k_1 := k_1 + 1$$

$$\square \ b_1[k_1] > b_2[k_2] \rightarrow k_2 := k_2 + 1 \ \mathbf{od}$$

- Before discussing the possible outcomes, let's modify K . First, let's merge some arms, since each $k_i := k_i + 1$ appears twice. And we have:

$$K_1 \equiv$$

$$k_0 := 0; k_1 := 0; k_2 := 0;$$

$$\mathbf{do} \ b_0[k_0] < b_1[k_1] \vee b_0[k_0] < b_2[k_2] \rightarrow k_0 := k_0 + 1$$

$$\square \ b_0[k_0] > b_1[k_1] \vee b_1[k_1] < b_2[k_2] \rightarrow k_1 := k_1 + 1$$

$$\square \ b_0[k_0] > b_2[k_2] \vee b_1[k_1] > b_2[k_2] \rightarrow k_2 := k_2 + 1 \ \mathbf{od}$$

- K_1 is already pretty good, but we can still simplify each guard. For example, in the first arm, we only need $b_0[k_0]$ less than something to increase k_0 , so we can come up with:

$K_2 \equiv$

```

 $k_0 := 0; k_1 := 0; k_2 := 0;$ 
do  $b_0[k_0] < b_1[k_1] \rightarrow k_0 := k_0 + 1$ 
    $\square b_1[k_1] < b_2[k_2] \rightarrow k_1 := k_1 + 1$ 
    $\square b_2[k_2] < b_0[k_0] \rightarrow k_2 := k_2 + 1$  od

```

If all three guards are False, then we have $b_0[k_0] \geq b_1[k_1] \geq b_2[k_2] \geq b_0[k_0]$ which implies an equality among all three values.

- Does $\perp_e \in M(K_2, \sigma)$?

It is possible. We can increase the value some k_i from $n - 1$ to n and have runtime error in the next iteration.

- Now, let's take care of $\perp_e$. We can simply add some bounds checks in each guard, then:

$K_3 \equiv$

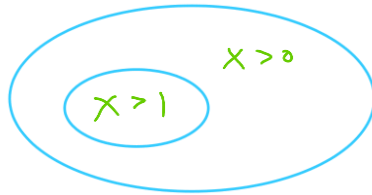
```

 $k_0 := 0; k_1 := 0; k_2 := 0;$ 
do if  $k_0 \geq n \vee k_1 \geq n$  then  $F$  else  $b_0[k_0] < b_1[k_1]$  fi  $\rightarrow k_0 := k_0 + 1$ 
    $\square$  if  $k_1 \geq n \vee k_2 \geq n$  then  $F$  else  $b_1[k_1] < b_2[k_2]$  fi  $\rightarrow k_1 := k_1 + 1$ 
    $\square$  if  $k_2 \geq n \vee k_0 \geq n$  then  $F$  else  $b_2[k_2] < b_0[k_0]$  fi  $\rightarrow k_2 := k_2 + 1$  od

```

Strength of Predicates

- A predicate can be considered as the collection of states that satisfy it; in other words, we can say $p = \{ \tau \in \Sigma \mid \tau \models p \}$.
 - For example, we can say $\{x = 1, y = 2\} \models x + y \leq 4$ and we can also say $\{x = 1, y = 2\} \in x + y \leq 4$.
- A predicate is **stronger** if it is “more restricted”. For example, we can consider $x > 1$ is stronger than $x > 0$.
 - If we consider these predicates as collection of states, then $x > 1$ is a subcollection of $x > 0$.



- A predicate is **weaker** if it is “less restricted”. For example, we can consider $x > 0$ is weaker than $x > 1$.
 - If we consider these predicates as collection of states, then $x > 0$ is a super-collection of $x > 1$.
 - If predicates $p \Rightarrow q$, then p is **stronger** than q and q is **weaker** than p .
 - Technically, it should be “stronger than or equal to” and “weaker than or equal to”. Because if $p \Leftrightarrow q$, then are equally strong.
2. What is the strongest predicate and what is the weakest predicate?
- A predicate is weaker when it is less restricted, and more states can satisfy it. There is a predicate that all states satisfy it, which is T ; so T is the weakest predicate.
 - On the hand, F is so restricted such that no state can satisfy it; so F is the strongest predicate.

Correctness Triples

- A **correctness triple** (a.k.a. “Hoare triple,” after C.A.R. Hoare; or usually simplified to “**triple**”), written as $\{p\} S \{q\}$ is a program S plus its specification predicates p and q .
 - The **precondition** p (not “ $\{p\}$ ”) describes the collection of states that we want to execute S in.
 - The **postcondition** q (not “ $\{q\}$ ”) describes the collection of states we expect S terminates in.
 - *Informally*, a triple $\{p\} S \{q\}$ means “if program S runs in a state that satisfies p , then we expect the execution of S terminates in some state (or states) satisfies q ”.

Here are some examples of correctness triples:

- $\{x \leq 2\} x := x + 3 \{x < 6\}$
- $\{x \geq 0\} \text{skip} \{y^2 \leq x < (y + 1)^2\}$
- $\{x > 100\} \text{while } x > 0 \text{ do } x := x + 1 \text{ od } \{x < 0\}$

A tripe can “make no sense”: the execution of S in a state satisfying p can never ends in some state satisfying q . So here, let us understand the satisfaction and validity of a triple.

(Satisfaction and Validity under Total Correctness)

- Let σ be a state proper for S . The triple $\{p\} S \{q\}$ is **totally correct in σ** (or σ **satisfies the triple under total correctness**), written as $\sigma \models_{tot} \{p\} S \{q\}$, if and only if it is the case that “if σ satisfies p , then the execution of S in σ always terminates successfully (without error) in states satisfying q ”.
- In other words, $\sigma \models_{tot} \{p\} S \{q\}$ means “if $\sigma \models p$ then $M(S, \sigma) \models q$ ”.

3. True or False.

- $\{x = -5\} \models_{tot} \{x > 0\} x := x + 1 \{x > 0\}$
True. Since $\{x = -5\}$ doesn’t satisfy the precondition $x > 0$, so the triple is trivially satisfied.
 - $\{x = 0\} \models_{tot} \{1/x \geq 0\} x := x + 1 \{x < 0\}$
True. Since $\{x = 0\}$ doesn’t satisfy the precondition $1/x \geq 0$, so the triple is trivially satisfied.
 - $\{x = -1\} \models_{tot} \{T\} x := x + 1 \{x \geq 0\}$
True. Since $\{x = -1\}$ satisfies the precondition T , so we need to execute $x := x + 1$, and $M(x := x + 1, \{x = -1\}) = \{x = 0\}$, and it satisfies the postcondition $x \geq 0$.
 - $\{x = -5\} \models_{tot} \{x \leq 0\} x := x + 1 \{x \geq 0\}$
False. Since $\{x = -5\}$ satisfies the precondition $x \leq 0$, so we need to execute $x := x + 1$, and $M(x := x + 1, \{x = -5\}) = \{x = -4\}$, and it doesn’t satisfy the postcondition $x \geq 0$.
 - $\{x = 0\} \models_{tot} \{x \leq 0\} x := 1/x \{x \geq 0\}$
False. Since $\{x = 0\}$ satisfies the precondition $x \leq 0$, so we need to execute $x := 1/x$, and $M(x := 1/x, \{x = 0\}) = \{\perp_e\}$, and it doesn’t satisfy the postcondition $x \geq 0$.
- From the above examples, we can see that “ $\sigma \models_{tot} \{p\} S \{q\}$ ” might not give us information about the execution of S in σ . But on the other hand, “ $\sigma \not\models_{tot} \{p\} S \{q\}$ ” shows that $\sigma \models p$ and the execution of S in σ doesn’t end in states satisfying q .

- The triple $\{p\} S \{q\}$ is **totally correct** (or the triple is **valid under total correctness**) if and only if $\sigma \models_{tot} \{p\} S \{q\}$ for all $\sigma \in \Sigma$ (by default, we consider only those states proper for S). We write $\models_{tot} \{p\} S \{q\}$.

- $\not\models_{tot} \{p\} S \{q\}$ means the triple is invalid: there exists some state σ such that $\sigma \not\models_{tot} \{p\} S \{q\}$

4. True of False

- a. $\models_{tot} \{x > 0\} x := x + 1 \{x > 0\}$ True
- b. $\models_{tot} \{x > 0\} x := x - 1 \{x > 0\}$ False, we can find $\{x = 1\} \not\models \{x > 0\} x := x - 1 \{x > 0\}$

(Satisfaction and Validity under Partial Correctness)

- Let σ be a state proper for S . The triple $\{p\} S \{q\}$ is **partially correct** in σ (or σ satisfies the triple under partial correctness), written as $\sigma \models \{p\} S \{q\}$, if and only if it is the case that "if σ satisfies p , then if the execution of S in σ can terminate without an error, it terminates in states satisfying q ".
 - In other words, $\sigma \models \{p\} S \{q\}$ means "if $\sigma \models p$ then $M(S, \sigma) - \perp \models q$ ".
- The triple $\{p\} S \{q\}$ is **partially correct** (or the triple is **valid under partial correctness**) if and only if $\sigma \models \{p\} S \{q\}$ for all $\sigma \in \Sigma$ (by default, we consider only those states proper for S). We write $\models \{p\} S \{q\}$.

5. True or False.

- a. $\{x = -5\} \models \{x > 0\} x := x + 1 \{x > 0\}$ True. Precondition is not satisfied.
- b. $\{x = -1\} \models \{T\} x := x + 1 \{x \geq 0\}$ True.
- c. $\{x = -5\} \models \{x \leq 0\} x := x + 1 \{x \geq 0\}$ False.
- d. $\{x = 0\} \models \{x \leq 0\} x := 1/x \{x \geq 0\}$
True. Since $\{x = 0\}$ satisfies the precondition $x \leq 0$, so we need to execute $x := 1/x$, and $M(x := 1/x, \{x = 0\}) - \perp = \emptyset$, and it satisfies the postcondition $x \geq 0$.

6. If $\sigma \models p$ and $M(S, \sigma) = \{\perp\}$, then:

- a. Does $\sigma \models_{tot} \{p\} S \{q\}$? No, $M(S, \sigma) \not\models q$.
- b. Does $\sigma \models \{p\} S \{q\}$? Yes, $M(S, \sigma) - \perp \models q$.

- The difference between two correctness is whether we accept that executing S in σ ends with $\perp$. We can say: $\sigma \models_{tot} \{p\} S \{q\}$ means " $\sigma \models \{p\} S \{q\}$ and $\perp \notin M(S, \sigma)$ ".

7. True or False:

- a. $\models_{tot} \{F\} S \{q\}$ True, nothing can satisfy the precondition.
- b. $\models_{tot} \{p\} S \{T\}$ False, it is not true for some $\sigma \models p$ such that $\perp \in M(S, \sigma)$
- c. $\models \{F\} S \{q\}$ True, nothing can satisfy the precondition.
- d. $\models \{p\} S \{T\}$ True, for any state $\sigma \models p$, we have $M(S, \sigma) - \perp \models T$

8. Let $W \equiv \mathbf{while } k \neq 0 \mathbf{ do } k := k - 1 \mathbf{ od}$. Decide true or false for each of the following validity.

- a. $\models_{tot} \{k \geq 0\} W \{k = 0\}$
True. If $\sigma \models k \geq 0$, then the execution of W in σ will terminate in a state where $k = 0$.
- b. $\models_{tot} \{k = -1\} W \{k = 0\}$ False. W will diverge in a state that satisfies $k = -1$.
- c. $\models \{k = -1\} W \{k = 0\}$ True.
- d. $\models_{tot} \{T\} W \{k = 0\}$ False. If $k < 0$ then W diverges.
- e. $\models \{T\} W \{k = 0\}$ True. If $k < 0$ then W diverges or else W ends with $k = 0$.